Kasun Abaywardana

NODEJS API ASESSMENT

Fullstack Software Engineer - Web and GenAI - Tech Lead

1. Project Stack

Layer	Technology	Purpose
Runtime	Node.js 20 + TypeScript	Type compatibility
Framework	NestJS 11	Node JS Express Based Framework
ORM	Prisma 6	Type-safe DB access & schema management
Database	MySQL 8.0	Relational persistence
Validation	class-validator + class-transformer	DTO validation with decorators
API Docs	Swagger / OpenAPI 3 (@nestjs/swagger)	Interactive API documentation
Logging	nestjs-pino + pino-http	Structured JSON observability
Testing	Jest + @nestjs/testing	Unit testing (backend)
Containerise	Docker Compose	Local MySQL 8 + Adminer UI
Frontend	Next.js 16 + React 19 + Tailwind CSS	Admin UI
Deploy (BE)	Railway	PaaS — auto deploy from GitHub
Deploy (FE)	Vercel	Next.js hosting

2. Project Structure

The monorepo contains two independently deployable applications under backend/ and frontend/ with shared root-level npm scripts for concurrent local development.

Backend follows a strict layered architecture inside src/:

- Controller → validates HTTP input, delegates to service, shapes response
- Service → all business logic, cross-entity validation, throws NestJS HTTP exceptions
- Repository → all Prisma ORM queries
- Module → NestJS DI wiring per feature (registrations, students, teachers)
- Common → global exception filter, shared DTOs, error response shape

Key modules:

- registrations — register, common-students, suspend, retrieve-for-notifications
- students — upsert, suspend, paginated list with teachers(Additional API For FE)
- teachers — upsert, findByEmail
- prisma — singleton PrismaService (OnModuleInit / OnModuleDestroy lifecycle)
- health — GET /health liveness probe (For Railway Deployment)

3. Database & Migration Strategy

Schema

Three tables with cascade-delete foreign keys:

Table	Key Columns	Notes
teachers	id (PK), email UNIQUE	Auto-incremented PK, unique email index
students	id (PK), email UNIQUE, isSuspended BOOL	Suspension flag, unique email index
teacher_students	teacherId + studentId (composite PK)	Junction table — CASCADE on both FKs

Migration Strategy

- prisma/schema.prisma defines models declaratively; Prisma generates both the SQL migration and the TypeScript client
- Migrations live in prisma/migrations/ and are committed to version control
- Local dev: npx prisma migrate dev — applies pending migrations and regenerates the client
- Production: npm run deploy runs prisma migrate deploy then starts the server — fully automated via Railway deploy hook
- Docker Compose provides a local MySQL 8 instance with a health-check gate so the backend only starts after the DB is ready

4. API Endpoints & Swagger Documentation

All endpoints are documented with @nestjs/swagger decorators — request bodies, query params, and all possible response shapes (200 / 204 / 400 / 404) are described inline in the code. The Swagger UI is auto-generated at /api/docs.

Method	Path	Description	Response
POST	/api/register	Register students to a teacher (upsert)	204 No Content
GET	/api/commonstudents	Students common to all queried teachers	200 { students[] }
POST	/api/suspend	Suspend a student by email	204 No Content
POST	/api/retrievefornotifications	Recipients: registered + @mentioned students	200 { recipients[] }
GET	/api/students	Paginated list of all students with teachers	200 paginated DTO
GET	/health	Liveness probe	200 { status: ok }

Live Swagger UI:

https://admin-school-assignment-production.up.railway.app/api/docs

5. Deployment Architecture

The application is deployed as two fully managed, independently scalable services:

Backend Folder Structure

```
backend/
├── prisma/
│   ├── schema.prisma             # Data models (Teacher, Student, TeacherStudent)
│   └── migrations/               # Versioned SQL migration files
│       └── 20260321073337_init/
│           └── migration.sql
├── src/
│   ├── main.ts                   # Bootstrap: CORS, validation pipe, Swagger, Pino logger
│   ├── app.module.ts             # Root module — wires LoggerModule, APP_FILTER, feature modules
│   │
│   ├── common/
│   │   ├── dto/
│   │   │   └── error-response.dto.ts          # Swagger error shape { message }
│   │   └── filters/
│   │       └── http-exception.filter.ts       # Global exception filter — logs 5xx, returns { message }
│   │
│   ├── prisma/
│   │   ├── prisma.module.ts       # Global PrismaModule
│   │   └── prisma.service.ts      # PrismaClient singleton with lifecycle hooks
│   │
│   ├── health/
│   │   └── health.controller.ts   # GET /health liveness probe
│   │
│   ├── teachers/
│   │   ├── teachers.module.ts
│   │   ├── teachers.service.ts    # upsertByEmail, findByEmail
│   │   ├── teachers.repository.ts # Prisma queries
│   │   └── entities/
│   │       └── teacher.entity.ts
│   │
│   ├── students/
│   │   ├── students.module.ts
│   │   ├── students.controller.ts # GET /api/students (paginated)
│   │   ├── students.service.ts    # upsert, suspend, paginated list — logs suspension
│   │   ├── students.repository.ts # Prisma queries
│   │   ├── dto/
│   │   │   ├── get-students-query.dto.ts
│   │   │   └── student-with-teachers.dto.ts
│   │   └── entities/
│   │       └── student.entity.ts
│   │
│   ├── registrations/
│   │   ├── registrations.module.ts
│   │   ├── registrations.controller.ts  # POST /api/register, GET /api/commonstudents,
│   │   │                                # POST /api/suspend, POST /api/retrievefornotifications
│   │   ├── registrations.service.ts     # Business logic + conflict validation + logging
│   │   ├── registrations.repository.ts  # ORM queries: common students, link teacher↔student
│   │   ├── dto/
│   │   │   ├── register.dto.ts
│   │   │   ├── suspend.dto.ts
│   │   │   ├── retrieve-notifications.dto.ts
│   │   │   ├── common-students-query.dto.ts
│   │   │   ├── common-students-response.dto.ts
│   │   │   └── notification-recipients-response.dto.ts
│   │   └── interfaces/
│   │       └── registration.interface.ts
│   │
│   └── __tests__/                 # Unit tests (Jest) — one spec per source file
│       ├── common/
│       │   └── http-exception.filter.spec.ts
│       ├── registrations/
│       │   ├── registrations.controller.spec.ts
│       │   ├── registrations.service.spec.ts
│       │   └── registrations.repository.spec.ts
│       ├── students/
│       │   ├── students.controller.spec.ts
│       │   ├── students.service.spec.ts
│       │   └── students.repository.spec.ts
│       └── teachers/
│           ├── teachers.service.spec.ts
│           └── teachers.repository.spec.ts
└── package.json
```

Backend → Railway

- Railway detects the backend/ subdirectory as the service root via railway.json
- Build step: nest build && prisma generate — compiles TypeScript and generates the Prisma client
- Start step: prisma migrate deploy && node dist/main — runs pending DB migrations then boots the server
- Railway provisions a managed MySQL 8 database plugin and injects DATABASE_URL automatically
- PORT is also auto-injected; ALLOWED_ORIGINS is set to the Vercel frontend URL for CORS
- Zero-downtime deploys on every push to main via GitHub integration

Frontend → Vercel

- Vercel imports the repository and uses frontend/ as the root directory
- BACKEND_URL env variable points to the Railway API URL; Next.js server-side API routes proxy requests to avoid CORS from the browser
- Edge-optimised CDN delivery for static assets; SSR for the admin page

Links

Service	URL
Frontend Live (Vercel)	https://admin-school-assignment.vercel.app
Backend API Live (Railway)	https://admin-school-assignment-production.up.railway.app
Swagger UI Live	https://admin-school-assignment-production.up.railway.app/api/docs

6. Unit Testing

71 tests across 9 test suites — all unit tests, zero external dependencies, no real database required. Every layer (controller → service → repository) is tested in isolation using Jest and @nestjs/testing with full NestJS dependency injection.

Test Considerations:

- PrismaService is mocked at the DI level — tests never touch a real database
- PinoLogger is mocked per-service using getLoggerToken() — logging calls are asserted without side effects
- Each suite calls jest.clearAllMocks() in beforeEach.

7. Observability (Structured Logging)

nestjs-pino (backed by Pino, the fastest Node.js logger) provides structured JSON logging in production and colourised pretty-print in development — zero configuration change required between environments.

What is logged automatically

- Every HTTP request and response: method, URL, status code, response time (pino-http middleware wired into LoggerModule)
- NestJS bootstrap messages (route registration, app start) replace the default NestJS console logger

PinoLogger is injected per-service using `@InjectPinoLogger(ServiceName.name)`, which makes logs request-scoped - every log line from a service call automatically carries the same reqId as the HTTP request log, enabling full request traceability without manual correlation IDs.

8. Other things in Backend

- Global exception filter
- Cross-table email validation
- Upsert-on-register : POST /api/register: teachers and students are upserted, and the teacher_students composite PK prevents duplicate links
- Parameterised ORM queries
- CORS guard : origin whitelist read from ALLOWED_ORIGINS env variable; unknown origins are rejected with a clear error, not silently passed
- Input validation : class-validator decorators on all DTOs with a custom exceptionFactory
- Extra endpoint : GET /api/students returns a paginated list of all students with their registered teachers and suspension status, useful for the admin FE

---

9. Frontend — React / Next.js Admin UI

A simple but complete single-page admin panel was built with Next.js 16 (App Router) and React 19 to demonstrate end-to-end API integration. The UI is deployed to Vercel and proxies all API calls through Next.js server-side route handlers to avoid CORS issues in the browser.

Components & Features

Component	API Used	What It Does
RegisterSection	POST /api/register	Form with teacher email + dynamic multi-email student input; success/error toast feedback
CommonStudentsSection	GET /api/commonstudents	Multi-teacher email input; displays intersection of registered students
NotificationsSection	POST /api/retrievefornotifications	Teacher + notification text input; displays eligible recipients
SuspendModal	POST /api/suspend	Modal dialog triggered from header; suspends a student by email
StudentsTableSection	GET /api/students	Paginated data table showing all students, their teachers, and suspension status

Technical Selections for Frontend

- Next.js App Router with "use client" components for interactive forms
- Tailwind CSS for utility-first responsive styling
- Shared TypeScript interfaces (types/index.ts) mirror the backend DTO shapes for type safety across the boundary
- Reusable UI primitives: Loader, Toast, Modal, EmailListInput shared across feature sections
- All API calls go through /app/api/* Next.js route handlers (server-side) — backend URL is never exposed to the browser